University of Southern Denmark

Department of Mathematics and Computer Science

DM575: Object-Oriented Programming

Pacman

Group T20:

Ehsan Sorousharzani
xx.x

Sebastian Juhl-Jensen
xx.x

Vincent Kirk Bonné
vibon25x student.sdu.dk

May 16, 2026



1 Introduction and Preliminaries

2 Design

DRY - Dont Repeat Yourself: MovingEntity indeholder den kode alle movingentities skal dele, så det ikke skrives flere gange

SOLID:

Single Responsibility Principle: Opdeling af model i Maze, Pacman etc.

Open/Closed Principle: Ghost abstract class indeholder basal Ghost logic, men udvides til andre behaviours ved inhreitance

Liskov Substitution Principle: Inky, Blinky etc. overrider kun scatter og target, med det samme forventet resultat, at finde et target at gå mod, ikke andre uforventede side effects.

Interface Segregation Principle: MovingEntity indeholder fx ikke ghost specifik kode, da andre entities ikke nødvendigvis har brug for det.

Dependency Inversion Principle: I stedet for at have en Inky, en Blinky etc. i Model, har vi en liste over Ghosts, som indeholder de forskellige typer.

Har vi brugt andre OO Principper?

3 Implementation

To build the project we use Maven and java. To run the project use the command

```
mvn javafx:run
```

.

We use java version 17.

Nok også noget andet.

3.1 View

3.2 Ghost behaviour

When we first implemented this we only had a single ghost to make testing easier.

In the original pacman they didnt use a real search algorithm as far as we could tell. They just chose the direction that would move them the closest to pacman. So that is the first thing we tried. The code looked like this.

```
public void goto_target(double targetX, double targetY, Maze maze) {
    Direction opposite = Direction.oppositeDirection(getDirection());
    double minDist = Double.POSITIVE_INFINITY;

    for (Direction d : Direction.values()) {
```

```
        if (d == Direction.NONE)
            continue;
        if (d == opposite)
            continue;
        double newx = Math.floor(x) + d.dx;
        double newy = Math.floor(y) + d.dy;
        if (maze.getTile(newx, newy) == Tile.Wall)
            continue;

        double xdiff = newx - targetX;
        double ydiff = newy - targetY;
        double dist = xdiff*xdiff + ydiff*ydiff;
        if (dist < minDist) {
            minDist = dist;
            setWantedDirection(d);
        }
    }
}
```

We didnt end up going for this in the end as the ghosts often made bad decisions.

In our final version we used the A star algorithm to find the fastest path. It is the standard version of astar taken from Wikipedia. With some minor changes to make it apply better to our needs like not being able to go backwards.

4 Quality assurance

5 Project management

The first phase of our work consisted of colaborating one how we should structure our code, so we started by brainstorming how the UML diagram should look. For equal share of work and for efficiency we devided the work up, so we simoultaneously worked on different part of the project in parallel. In more detail, we devided the work into: the graphical content of the game, implementation og the ghosts and the implementation of the gameModel where everything lives. We then explained our implementations to each other and cleared up any confusions about each others code.

In times of where someone has been stuck, we have been very good at helping each other out, to ensure no-one gets stuck in thier work. For colaboration we used GIT to push the changes that were made, and pull each others work. This tool helped us to see earlier versions of our code, when something was broken, or the code wouldn't compile.

6 Discussion